Week 2 Learning Report -- DINO from First Principles

1. DINO Architecture and Intuition

In Week 1 we used a pretrained DINOv2 model as a frozen feature extractor. This week our goal was to look inside the black box and build the core DINO algorithm (Caron et al., 2021) from scratch on a smaller scale so we can see how a Vision Transformer learns meaningful features without any labels.

The name DINO stands for self-DIstillation with NO labels. In standard knowledge distillation, we train a small student network to match the outputs of a larger, already-trained teacher network. DINO flips this around: both the student and the teacher have the exact same architecture (a Vision Transformer backbone followed by a 3-layer MLP projection head), and neither one starts with any labels or pretraining. Instead, the student learns from the teacher while the teacher is continuously built as a running average of the student.

In our Mini-DINO setup on CIFAR-10 (5,000 images, 32x32 resolution), we built a TinyViT with patch size 4x4 (giving 8x8 = 64 patches plus 1 CLS token), embedding dimension 192, 4 Transformer blocks, and 3 attention heads, followed by a projection head outputting a 256-dimensional vector.

2. The Student-Teacher Mechanism

During training, we pass different augmented crops of the same image through the student and the teacher. Both networks output a 256-dimensional vector of logits, which we convert into probability distributions P_s and P_t using a temperature-scaled softmax.

The training loss is the cross-entropy between the teacher's distribution and the student's distribution: H(P_t, P_s) = - sum(P_t * log(P_s)). Crucially, gradients only flow through the student network. We detach the teacher's outputs (using torch.no_grad()) so the teacher never gets updated by backpropagation directly.

Because both networks are trying to agree on representations of different regions of the same image, the student is forced to learn high-level semantic features (like object shape and category) rather than memorising pixel-level details.

3. EMA, Multi-Crop, Centering, and Sharpening

If we just trained two identical networks to match each other, they would immediately collapse -- either outputting the exact same constant vector for every image, or putting 100% probability on a single dimension. DINO avoids collapse using three mechanisms working together:

Momentum / EMA Teacher: Instead of training the teacher with gradients, we update its weights as an Exponential Moving Average of the student's weights: theta_t <- lambda * theta_t + (1 - lambda) * theta_s, where lambda follows a cosine schedule from 0.996 up to 1.0. Because lambda is close to 1, the teacher changes very slowly. It acts like a stable ensemble of past student models, giving the student a consistent target to chase.

Multi-Crop Augmentation: For every image, we generate 2 global crops (covering 40% to 100% of the image) and 4 local crops (covering 5% to 40% of the image). Only the global crops go to the teacher, while all 6 crops go to the student. When the student sees a tiny local patch (say, just the wheel of a car) and has to match the teacher's output from the whole car, it has to learn local-to-global context.

Centering and Sharpening: We maintain an exponential moving average center vector c of the teacher's outputs and subtract it before applying softmax: P_t = softmax((g_t(x) - c) / tau_t). Centering stops any single dimension from dominating across the batch, pushing outputs toward a uniform distribution. To balance this out, we use a much sharper temperature for the teacher (tau_t = 0.04) than for the student (tau_s = 0.1). Centering prevents collapse to a single dimension, while sharpening prevents collapse to a flat uniform distribution.

4. Experimental Observations

We trained Mini-DINO for 15 epochs on 5,000 CIFAR-10 images. First, we verified our EMA implementation by inspecting the outputs and weights after training: on a test batch, the student and teacher outputs had a mean absolute difference of around 0.05--0.15, and the parameter tensors differed slightly, confirming that the teacher was lagging smoothly behind the student rather than staying frozen or identical.

The training loss started near the theoretical entropy of a 256-way distribution and decreased steadily across the 15 epochs as the student learned to predict the teacher's sharpened global representations from local crops.

When we extracted the 192-dimensional CLS tokens from the TinyViT backbone and projected them to 2D with PCA and t-SNE, we saw clear structure: even with only 15 epochs and 5,000 images, categories with distinct visual backgrounds (like airplanes/ships with blue skies and water, or frogs/deer with natural greenery) started grouping together without a single label being used during training.

Even more interesting were the self-attention maps from the final Transformer layer. By plotting the attention weights from the CLS token to the 8x8 patch grid, we saw that the model naturally attends to the foreground object (the animal or vehicle) and ignores the background, with different attention heads specialising in different parts of the object.

5. Ablation Study: With vs Without EMA

To test how much the momentum teacher actually matters, we ran an ablation where we disabled EMA (use_ema=False) and simply copied the student's weights directly into the teacher after every step (equivalent to lambda = 0).

The difference was immediately visible. Without EMA, the teacher changes just as fast as the student, so the target distribution shifts abruptly at every gradient step. While centering and sharpening still prevented total numerical NaN collapse, the loss curve behaved much more erratically and converged to a worse plateau.

In the t-SNE comparison, the embeddings trained without EMA were noticeably more mixed and diffuse compared to the EMA run. This confirmed the main takeaway from the DINO paper: having a slowly-evolving momentum teacher is essential for stable self-distillation and for learning clean semantic representations.